

**PHRASE MINING****phrase\_miner**

Iterative k-mer mining (k=25→3) with masking & de Bruijn extension.

```
-i input -o dir -K k_max -k k_min -n freq -t tinyids -D deBruijn -S subsumption --dedup
```

**phrase**

Original n-gram detection (simpler, single-pass).

```
-i -o --fields --min-words --min-ids --lemmatize --verbatim
```

**phrase\_builder**

K-mer frequency visualization charts.

```
-i -m model -o
```

**phrase\_grouper**

Bottom-up family analysis (prefix/suffix/infix).

```
-i -o -K -k --min-family-size --assignment
```

**INSTRUMENT EXTRACTION****instrument\_miner**

Extract “as part of <Instrument>” patterns.

```
-i -o -a abbreviations -s supplementary -d families --family-summary
```

**batch\_expand\_abbreviations**

Iterate abbreviation discovery for full instrument names.

```
-i --abbreviations FILE -o --k-max --k-min --top-phrases
```

**LLM CLASSIFICATION****llm\_classify**

Multi-provider LLM phrase classification.

```
-i dir -m {instrument,temporal,instrument_family} -o dir -providers {claude,openai,google} --batch-size --context-window --dry-run
```

**DISCOVERY & STRIPPING****strip\_discover**

Find pattern occurrences in CDE text fields (verbatim + variants).

```
-i -m -o -p patterns -e expand -b bare -a abbreviations -w workers
```

**strip\_phrases**

Apply curated patterns to remove matching text from fields.

```
-i -m -o -p / --phrases -I ignore-case -B word-boundary -T trace --clean-remnants
```

**strip\_analyze**

Detect conflicts, overlaps, and false negatives.

```
-i -o -p --analyze-conflicts --analyze-false-negatives
```

**strip\_html**

Remove HTML markup from CDE text fields.

```
--input --output -m --pretty --tables
```

**DIAGNOSTIC****diagnose\_strip**

Find remaining anchor patterns after stripping (residuals).

```
-i -m -o --anchors --fields --min-count --suggest-patterns --emit-tinyids
```

**DATA CLEANING****fix\_underscores**

Fix Pydantic-incompatible underscore-prefixed keys.

```
-i -o --prefix CHAR --depth N
```

**PATTERN UTILITIES: ANALYSIS**

Multi-mode tool — one mode per invocation.

**Coalesce Variants**

```
-c FILE -o OUT — subsumption + prefix trie
--coalesce-report FILE --min-prefix-tinyids N
```

**Merge Patterns**

```
-M FILE FILE -o OUT — deduplicate/merge TSVs
--merge-pattern-column --merge-tinyids-column
```

**Field Analysis**

```
-A FILE -i JSON -m MODEL -o OUT
--min-field-count N --min-tokens N --exclude-patterns
```

**Validate Subsumption**

```
-v FILE -i JSON -o OUT -w workers
```

**PATTERN UTILITIES (CONT.)****Temporal Seeds**

```
-T -o OUT — expand 25 seeds → ~2,100 variants
```

**Expand Verbatim**

```
-e FILE -o OUT — generate pattern variants
```

**TSV Editor**

```
--edit FILE --port N — browser-based editor
```

**Supplementary Management**

```
--harvest-to-supplementary FILE — auto-ingest
--promote-supplementary — local → global
--clean-local — remove promoted entries
```

**REPORTING & ANALYSIS****discovery\_report**

Markdown summary of a pipeline execution.

```
-d dir -p {instrument,phrase} -o output --version -i json
```

**pipeline\_report**

Comprehensive multi-phase execution report.

```
-s state_file / -d dir -o -p {1,2,3,4} --title --version
```

**strip\_report**

Quality report for stripped outputs (remnant detection).

```
-d dir -o -i json --version --no-temporal-scan
```

**recall\_analyze**

Detection recall vs ground-truth patterns.

```
-i -m -o -F pattern_file --pipeline-output -r report --suggest-patterns
```

**DATA EXTRACTION****subset**

Extract CDE records by tinyld or text content.

```
-i -o -m -l id_list / -L id_file -t text --regex -x exclude
```

**extract\_embed**

Extract fields for transformer embeddings.

```
-i -m -o --path-file --output-format {json,csv,tsv} --concatenate
```

**count**

Count field occurrences and unique values.

```
-i --fields -o --match-type {non_null,null,fixed,regex} --group-by
```

**lemma\_fasta**

Lemmatized text → FASTA format for sequence tools.

```
-i -m -o -l / -L --remove-stopwords
```

**WORKFLOW ORCHESTRATION****Run a Pipeline**

```
cde-analyzer workflow run FILE.yaml \
-s input_json=data.json \
-s output_dir=output/ \
--dry-run # validate only
--from-step N # partial re-run
```

**Resume / Status / Copy**

|                 |                               |
|-----------------|-------------------------------|
| workflow resume | Continue after checkpoint     |
| workflow status | Show pipeline progress ( -v ) |
| workflow list   | Available workflow YAML files |
| workflow copy   | Copy template to local dir    |

**SHORT OPTIONS: I/O & MINING**

| Short | Long           | Meaning         |
|-------|----------------|-----------------|
| -i    | --input        | Input file      |
| -o    | --output       | Output file/dir |
| -m    | --model        | Pydantic model  |
| -f    | --fields       | Field paths     |
| -p    | --pattern-list | Pattern file    |
| -w    | --workers      | Parallelism     |
| -K    | --k-max        | Max k-mer len   |
| -k    | --k-min        | Min k-mer len   |
| -n    | --freq-min     | Min frequency   |
| -t    | --min-tinyids  | Min CDE count   |

**SHORT OPTIONS: FLAGS**

| Short | Long             | Meaning          |
|-------|------------------|------------------|
| -S    | --subsumption    | Subsumption on   |
| -D    | --debruijn       | De Bruijn on     |
| -I    | --ignore-case    | Case-insensitive |
| -B    | --word-boundary  | Word boundary    |
| -T    | --trace          | Trace / temporal |
| -M    | --merge          | Merge TSVs       |
| -C    | --coalesce       | Coalesce         |
| -A    | --field-analysis | Field counts     |
| -V    | --validate       | Validate subs.   |
| -e    | --expand         | Expand variants  |

Also: -x exclude, -l / -L id-list/file, -C case, -r report, -g ground-truth

**INPUT/OUTPUT PATTERNS**

| Pattern         | Description                  |
|-----------------|------------------------------|
| -i FILE -o FILE | Single input → single output |
| -i FILE -o DIR  | Input → directory of outputs |
| -p TSV -i JSON  | Patterns + source data       |
| -d DIR          | Output directory (reporting) |
| -s FILE         | Workflow state file          |

Models: -m CDE (default), -m Form, -m EmbedText

**AVAILABLE WORKFLOWS**

| File                     | Ph | Steps                 |
|--------------------------|----|-----------------------|
| instrument_pipeline.yaml | 1  | 9 (mine→strip→sanity) |
| phrase_pipeline.yaml     | 2  | 7 (mine→strip→report) |
| branching_strip.yaml     | 3  | 10 (5-way branch)     |

**COMMON INVOCATION**

```
# Full pipeline
cde-analyzer workflow run \
  instrument_pipeline.yaml \
  -s input_json=allcode.json

# Manual single-step
cde-analyzer phrase_miner \
  -i data.json -o out/ \
  -K 25 -k 3 -n 5 -t 3
```

cde-analyzer <command> --help for full options.